

UFHB – Département Informatique

L2 – Systèmes Embarqués et IoT

TP 5A

Connectivité IoT – Simulation ESP32 + MQTT

Wokwi · broker.hivemq.com · MQTTX

Durée : 4 heures | Niveau : L2 Informatique

Prérequis : TP0 à TP4 validés (Tinkercad)

Objectif pédagogique

Ce TP marque le passage de la couche hardware (Arduino/Tinkercad) vers la couche service (IoT pipeline).

À l'issue de cette séance, chaque étudiant aura :

- Simulé un ESP32 connecté au Wi-Fi sous Wokwi
- Publié des données capteurs via le protocole MQTT sur un broker public
- Observé et filtré les messages en temps réel dans MQTTX ou HiveMQ Web Client
- Appliqué un filtre Edge : seules les anomalies sont transmises (principe Edge Computing)
- Compris le rôle des composants L3 : Mosquitto, InfluxDB, Grafana

Ce TP est le pont pédagogique vers le cours L3 – Création de Services Réseaux et IoT

1. Contexte et positionnement

Jusqu'ici, vos programmes Arduino communiquaient localement via UART (Serial Monitor). Dans un vrai système IoT, ces données doivent être **transmises à un broker** qui les redistribue à des abonnés (dashboards, bases de données, alertes). C'est précisément ce que vous allez simuler.

Couche	Ce que vous avez vu / ferez
L2 – TP0→TP4 (Tinkercad)	Hardware : GPIO, ADC, UART, I ² C, capteurs analogiques
L2 – TP5A (ce TP – Wokwi)	Connectivité : ESP32, Wi-Fi simulé, MQTT Publish
Projet – Service stack	Infrastructure : Mosquitto, InfluxDB, Node-RED, Grafana

Pourquoi l'ESP32 et pas l'Arduino Uno ?

L'Arduino Uno (ATmega328P) n'a pas de Wi-Fi intégré. L'ESP32 (Xtensa LX6) intègre Wi-Fi + Bluetooth.

En L2, on l'utilise en simulation Wokwi pour sa connectivité native — sans matériel physique.

En projet, vous déploierez ce même code sur ESP32 physique connecté à votre stack Docker locale.

2. Outils et configuration

2.1 Wokwi – Simulateur ESP32

Wokwi (<https://wokwi.com>) est un simulateur en ligne qui émule l'ESP32 avec connectivité Wi-Fi simulée via un pont WebSocket. Il supporte Arduino IDE, MicroPython et PlatformIO.

Action	Détail
Créer un projet	→ New Project → ESP32 (Arduino)
Ajouter un capteur	Bibliothèque de composants à droite (drag & drop)
Activer le Wi-Fi simulé	Automatique — le simulateur injecte une connexion virtuelle
Bibliothèque MQTT	PubSubClient (disponible dans le gestionnaire de bibliothèques Wokwi)
Partager le circuit	Bouton Share → lien public à remettre à l'enseignant

2.2 MQTTX – Client de visualisation

MQTTX (<https://mqttx.app>) est un client MQTT graphique disponible en version web et desktop. Il permet de s'abonner à un topic et d'observer les messages en temps réel.

Configuration MQTTX pour ce TP

Broker host : broker.hivemq.com

Port : 1883 (TCP non chiffré) ou 8883 (TLS)

Client ID : mqttx_etudiant_VOTRENOM (unique !)

Topic (sub) : ufhb/<votre_login>/capteurs/#

QoS : 1

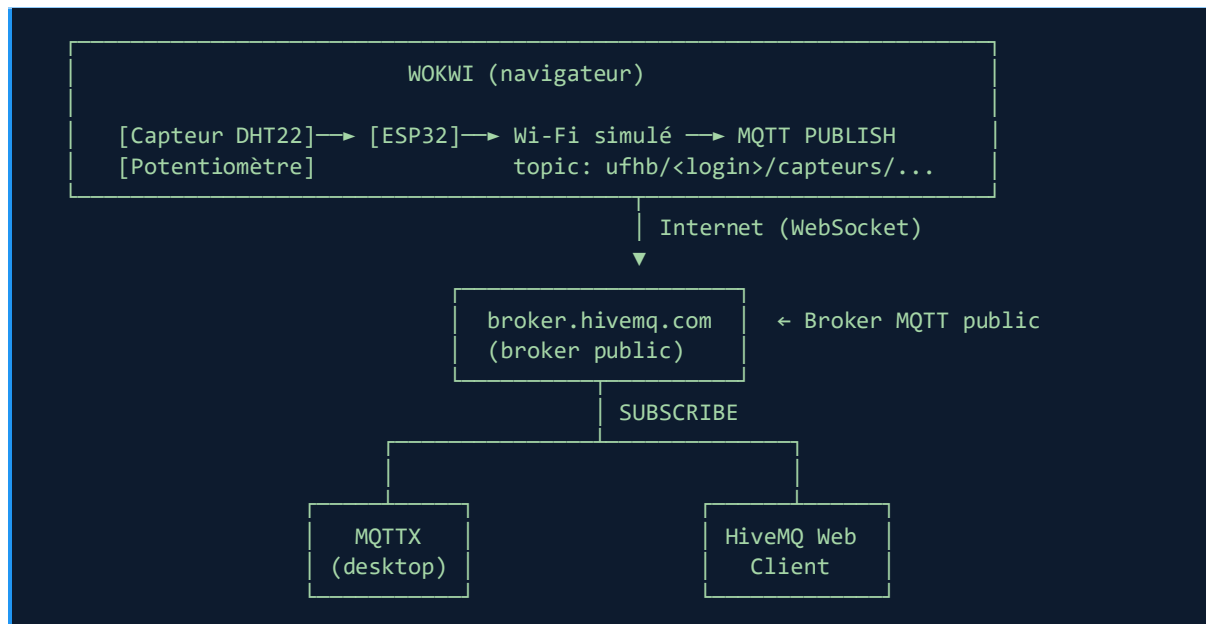
2.3 HiveMQ Web Client (alternative sans installation)

Si MQTTX n'est pas installable, utiliser <https://www.hivemq.com/demos/websocket-client/>

- Host : broker.hivemq.com | Port : 8000 (WebSocket)
- Topic à souscrire : ufhb/<votre_login>/capteurs/#

3. Architecture du système simulé

Le schéma suivant décrit le flux de données de ce TP :



En Projet, ce broker public sera remplacé par :

- Mosquitto (broker local dans Docker) → port 1883
- Node-RED (souscrit à Mosquitto, insère dans InfluxDB)
- InfluxDB (base de données temporelles) → port 8086
- Grafana (dashboard temps réel) → port 3000

Le code ESP32 ne changera que l'adresse du broker (IP locale au lieu de broker.hivemq.com).

4. Partie A – Circuit et premier PUBLISH (1h)

4.1 Montage Wokwi

Créez un nouveau projet ESP32 sur Wokwi et assemblez le circuit suivant :

Composant	Broche ESP32
DHT22 (température & humidité)	GPIO 15 (DATA) 3.3V GND
Potentiomètre 10kΩ (simulateur luminosité)	GPIO 34 (ADC1_CH6) 3.3V GND
LED rouge (alerte)	GPIO 2 GND (via 220Ω)

Attention – spécificités ESP32 vs Arduino

Les broches ADC de l'ESP32 mesurent de 0 à 3.3V (non 5V) — résolution 12 bits (0–4095).
 GPIO 34, 35, 36, 39 sont en entrée uniquement (input-only) — parfaits pour l'ADC.
 La bibliothèque DHT pour ESP32 est 'DHT sensor library' de Adafruit.
 PubSubClient nécessite une instance WiFiClient passée au constructeur.

4.2 Code de base – Connexion Wi-Fi + premier PUBLISH

Copiez et adaptez ce code dans l'éditeur Wokwi. Lisez chaque commentaire attentivement.

```
#include <WiFi.h>
#include <PubSubClient.h>
#include <DHT.h>

// — Configuration réseau (Wi-Fi simulé Wokwi) —
const char* SSID      = "Wokwi-GUEST"; // SSID fixe pour simulation
const char* PASSWORD = "";           // Pas de mot de passe en simulation

// — Configuration MQTT —
const char* BROKER    = "broker.hivemq.com";
const int   PORT      = 1883;
// Remplacez LOGIN par votre identifiant étudiant UFHB
const char* LOGIN     = "etudiant_UFHB_12345";

// — Topics de publication —
char topicTemp[64]; // ufhb/<login>/capteurs/temperature
char topicHum[64];  // ufhb/<login>/capteurs/humidite
char topicLux[64];  // ufhb/<login>/capteurs/luminosite
char topicAlerte[64]; // ufhb/<login>/capteurs/alertes

// — Capteurs —
#define DHT_PIN  15
#define DHT_TYPE DHT22
#define LUX_PIN  34
#define LED_PIN  2
DHT dht(DHT_PIN, DHT_TYPE);

WiFiClient  wifiClient;
PubSubClient mqtt(wifiClient);

void setup() {
  Serial.begin(115200);
  pinMode(LED_PIN, OUTPUT);
  dht.begin();

  // Construction des topics avec le login
  sprintf(topicTemp, sizeof(topicTemp), "ufhb/%s/capteurs/temperature", LOGIN);
  sprintf(topicHum,  sizeof(topicHum),  "ufhb/%s/capteurs/humidite", LOGIN);
  sprintf(topicLux,  sizeof(topicLux),   "ufhb/%s/capteurs/luminosite", LOGIN);
  sprintf(topicAlerte, sizeof(topicAlerte), "ufhb/%s/capteurs/alertes", LOGIN);

  connecterWifi();
  mqtt.setServer(BROKER, PORT);
  mqtt.setCallback(messageRecu); // optionnel pour ce TP
  connecterMQTT();
}

void loop() {
  if (!mqtt.connected()) connecterMQTT();
  mqtt.loop();

  // Lecture des capteurs
  float temp = dht.readTemperature();
  float hum  = dht.readHumidity();
  int  raw  = analogRead(LUX_PIN);
  int  lux  = map(raw, 0, 4095, 0, 100); // 0-100%
```

```

    if (isnan(temp) || isnan(hum)) {
        Serial.println("[ERR] Lecture DHT22 échouée");
        return;
    }

    publierDonnees(temp, hum, lux);
    delay(5000); // Publication toutes les 5 secondes
}

// — Fonctions de connexion —————
void connecterWifi() {
    Serial.printf("[WiFi] Connexion à %s", SSID);
    WiFi.begin(SSID, PASSWORD);
    while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
    Serial.printf("\n[WiFi] Connecté - IP : %s\n", WiFi.localIP().toString().c_str());
}

void connecterMQTT() {
    while (!mqtt.connected()) {
        String clientId = "esp32_ufhb_" + String(LOGIN);
        if (mqtt.connect(clientId.c_str())) {
            Serial.println("[MQTT] Connecté au broker");
        } else {
            Serial.printf("[MQTT] Échec rc=%d - nouvelle tentative dans 3s\n", mqtt.state());
            delay(3000);
        }
    }
}

void publierDonnees(float temp, float hum, int lux) {
    char payload[64];
    snprintf(payload, sizeof(payload), "%.1f", temp);
    mqtt.publish(topicTemp, payload, true); // retain=true
    snprintf(payload, sizeof(payload), "%.1f", hum);
    mqtt.publish(topicHum, payload, true);
    snprintf(payload, sizeof(payload), "%d", lux);
    mqtt.publish(topicLux, payload, true);
    Serial.printf("[PUBLISH] T=%.1f°C H=%.1f%% Lux=%d%%\n", temp, hum, lux);
}

void messageRecu(char* topic, byte* payload, unsigned int length) {
    // Sera utilisé dans la partie C (souscription)
    Serial.printf("[SUB] %s : ", topic);
    for (int i=0; i<length; i++) Serial.print((char)payload[i]);
    Serial.println();
}

```

Questions de compréhension – Partie A

1. Pourquoi le SSID est-il 'Wokwi-GUEST' et non un vrai nom de réseau ?
2. Quel est le rôle du paramètre retain=true dans mqtt.publish() ?
3. Que se passe-t-il si deux étudiants utilisent le même clientId sur le broker ?
4. À quoi sert mqtt.loop() appelé dans chaque itération de loop() ?

5. Partie B – Filtrage Edge : publier uniquement les anomalies (1h)

Le principe du **filtrage Edge (Edge Computing)** consiste à traiter les données localement, sur l'objet connecté, avant de les envoyer au réseau. On ne transmet que ce qui a de la valeur informationnelle — ici, les anomalies.

Paramètre	Seuil d'alerte
Température	> 35°C (canicule Abidjan)
Humidité	< 30% (air trop sec) ou > 85% (excès humidité)
Luminosité	< 10% (capteur obscurci / panne)

Ajoutez la fonction suivante à votre sketch Wokwi et modifiez loop() pour l'appeler :

```
// — Seuils d'alerte —————
const float SEUIL_TEMP_MAX = 35.0;
const float SEUIL_HUM_MIN  = 30.0;
const float SEUIL_HUM_MAX  = 85.0;
const int   SEUIL_LUX_MIN   = 10;

void evaluerEtPublierAlertes(float temp, float hum, int lux) {
    bool alerte = false;
    char msg[128];

    // — Vérification température —————
    if (temp > SEUIL_TEMP_MAX) {
        snprintf(msg, sizeof(msg),
            "{\"type\":\"TEMP_HAUTE\",\"valeur\":%.1f,\"seuil\":%.1f}", temp,
            SEUIL_TEMP_MAX);
        mqtt.publish(topicAlerte, msg);
        digitalWrite(LED_PIN, HIGH);
        alerte = true;
        Serial.printf("[ALERTE] Température %.1f°C > seuil %.1f°C\n", temp,
            SEUIL_TEMP_MAX);
    }

    // — Vérification humidité —————
    if (hum < SEUIL_HUM_MIN || hum > SEUIL_HUM_MAX) {
        snprintf(msg, sizeof(msg),
            "{\"type\":\"HUM_HORS_SEUIL\",\"valeur\":%.1f}", hum);
        mqtt.publish(topicAlerte, msg);
        alerte = true;
        Serial.printf("[ALERTE] Humidité hors seuil : %.1f%%\n", hum);
    }

    // — Vérification luminosité —————
    if (lux < SEUIL_LUX_MIN) {
        snprintf(msg, sizeof(msg),
            "{\"type\":\"LUX_FAIBLE\",\"valeur\":%d}", lux);
        mqtt.publish(topicAlerte, msg);
        alerte = true;
        Serial.printf("[ALERTE] Luminosité trop faible : %d%%\n", lux);
    }

    if (!alerte) {
        digitalWrite(LED_PIN, LOW);
        Serial.println("[OK] Valeurs dans les seuils - pas de publication d'alerte");
    }
}
```

```

}

// Dans loop(), remplacez l'appel à publierDonnees() par :
publierDonnees(temp, hum, lux);          // toujours publier les mesures
evaluerEtPublierAlertes(temp, hum, lux); // + alertes si anomalie

```

Manipulation à réaliser

1. Montez la température simulée au-delà de 35°C en modifiant le capteur DHT22 dans Wokwi.
→ Observer la LED qui s'allume et le message JSON sur le topic /alertes dans MQTTX.
2. Descendez la luminosité (potentiomètre) sous 10%.
→ Observer le message LUX_FAIBLE.
3. Notez la différence de fréquence : les mesures sont publiées toutes les 5s,
les alertes sont publiées immédiatement à chaque anomalie détectée.

Questions de compréhension – Partie B

5. Quel est l'avantage de traiter les seuils sur l'ESP32 plutôt que côté serveur ?
6. Dans quel scénario réel le filtrage Edge est-il indispensable ? (donnez un exemple ivoirien)
7. Le topic /alertes reçoit du JSON. Pourquoi JSON et non une simple valeur numérique ?

6. Partie C – Discussion guidée : vers le pipeline L3 (45 min)

À partir de ce que vous avez observé dans MQTTX, répondez aux questions suivantes en groupe.
L'enseignant guide la discussion.

Question
Qui stocke les données après le broker ?
Qui fait le lien entre Mosquitto et InfluxDB ?
Comment visualiser en temps réel ?
Que change-t-on dans le code ESP32 en L3 ?
Pourquoi Docker Compose ?

À rédiger (livrable de cette partie)

Schéma commenté (à la main ou Excalidraw) du pipeline complet :

ESP32 → Mosquitto → Node-RED → InfluxDB → Grafana

Pour chaque flèche : indiquer le protocole utilisé et le port.

Pour chaque bloc : indiquer son rôle en une phrase.

7. Livrables et grille d'évaluation

Modalités de rendu

Avant la fin de la séance, transmettre:

1. Lien de partage Wokwi (circuit + code)
2. Captures d'écran MQTTX : topics mesures + topic alertes
3. Fiche réponses (questions A et B)
4. Schéma pipeline L3 (photo ou PDF)

